Polynomials in PLONK example

Pushpa and Perplexity.ai

1. Example Circuit and Witness

We encode:

$$z = x^2 + xy + 3$$

Let $x = 2, y = 5$, so $z = 2^2 + 2 \cdot 5 + 3 = 17$.

Decompose into three gates:

$$\text{Gate 1: } a = x \cdot x$$
$$\text{Gate 2: } b = x \cdot y$$
$$\text{Gate 3: } z = a + b + 3$$

Witness assignments:

Gate	L	R	O	Operation
1	2	2	4	$a = x \cdot x$
2	2	5	10	$b = x \cdot y$
3	4	10	17	$z = a + b + 3$

2. Selector Polynomials

PLONK's general selector constraint per row:

$$q_L(i)L(i) + q_R(i)R(i) + q_M(i)L(i)R(i) + q_O(i)O(i) + q_C(i) = 0$$

For our three gates:

1. **Gate 1 (mul):** $O = L \cdot R$
   $\Rightarrow q_M = -q_O = 1, q_L = q_R = q_C = 0$
2. **Gate 2 (mul):** $O = L \cdot R$
   $\Rightarrow q_M = -q_O = 1, q_L = q_R = q_C = 0$
3. **Gate 3 (add+const):** $O = L + R + 3$
   $\Rightarrow q_M = 0, q_L = 1, q_R = 1, q_O = -1, q_C = 3$

So for each row (i):

i	$q_L(i)$	$q_R(i)$	$q_M(i)$	$q_O(i)$	$q_C(i)$
1	0	0	1	-1	0
2	0	0	1	-1	0
3	1	1	0	-1	3

Selector Lagrange Interpolation (Example for q_M)

Let us fix the evaluation domain as $n = 3$, so use field $\mathbb{F}_7$ (modulo 7 for smaller numbers) with primitive 3rd roots of unity. In $\mathbb{F}_7$, $2^3 = 1$, so $\omega = 2$ is a primitive 3rd root.

The x values for interpolation are: $\omega^0 = 1, \omega^1 = 2, \omega^2 = 4$.

For q_M: The values are $[1, 1, 0]$ at $[1, 2, 4]$. The interpolating polynomial is:

$$q_M(X) = q_M(1)\ell_1(X) + q_M(2)\ell_2(X) + q_M(4)\ell_3(X)$$

where

$$\ell_k(X) = \prod_{j \neq k} \frac{X - x_j}{x_k - x_j}$$

Thus,

$$\ell_1(X) = \frac{(X-2)(X-4)}{(1-2)(1-4)} = \frac{(X-2)(X-4)}{(-1)(-3)} = \frac{(X-2)(X-4)}{3}$$
$$\ell_2(X) = \frac{(X-1)(X-4)}{(2-1)(2-4)} = \frac{(X-1)(X-4)}{(1)(-2)} = -\frac{(X-1)(X-4)}{2}$$
$$\ell_3(X) = \frac{(X-1)(X-2)}{(4-1)(4-2)} = \frac{(X-1)(X-2)}{(3)(2)} = \frac{(X-1)(X-2)}{6}$$

So,

$$q_M(X) = 1 \cdot \ell_1(X) + 1 \cdot \ell_2(X) + 0 \cdot \ell_3(X) = \ell_1(X) + \ell_2(X)$$

$$\boxed{q_M(X) = \frac{(X-2)(X-4)}{3} - \frac{(X-1)(X-4)}{2}}$$

All coefficients are taken mod 7.

3. Wire Polynomials

Input values for columns:

$$L = [2, 2, 4], \quad R = [2, 5, 10], \quad O = [4, 10, 17]$$

Calculate each as a degree 2 polynomial so that $L(\omega^i) = L_i$.

As another example, $L(X) = 2 \cdot \ell_1(X) + 2 \cdot \ell_2(X) + 4 \cdot \ell_3(X)$. Same for $R(X), O(X)$. Coefficients are computed as above.

4. Permutation (Copy/Consistency) Polynomial Example

Step 1: Wire Position Table and Variable Occurrences

Index	Pos	Gate	Variable
0	L	1	x
1	R	1	x
2	O	1	a
3	L	2	x
4	R	2	y
5	O	2	b
6	L	3	a
7	R	3	b
8	O	3	z

Let's build the permutation: wherever the same variable occurs, create a cycle.

- x in (0,L,1), (1,R,1), (3,L,2): cycle these three.
- y in (4,R,2): appears just once, maps to itself.
- a in (2,O,1), (6,L,3): swap.
- b in (5,O,2), (7,R,3): swap.
- z in (8,O,3): only once.

So define mapping σ on index:
(indices: $0 \to 1, 1 \to 3, 3 \to 0$), ($2 \leftrightarrow 6$), ($5 \leftrightarrow 7$), ($4 \to 4$), ($8 \to 8$).

Step 2: Permutation Polynomial Construction

We want each row of each wire assignment (e.g., L wire) to know "where is the next occurrence of this logical variable?". We can encode this via three polynomials, $S_L(X), S_R(X), S_O(X)$. For example, at domain point ω^0 (first gate), the **Left** wire x at (0,L,1) is mapped under the permutation to (1,R,1), which in our enumeration is index 1, corresponding to ω^1.

So for each gate i, Let's break it down for each column:

- Left wire polynomial (S_L): entries at $i = 0, 3, 6$
- Right wire polynomial (S_R): entries at $i = 1, 4, 7$
- Output wire polynomial (S_O): entries at $i = 2, 5, 8$

So,

$$
\begin{aligned}
S_L(\omega^0) &= \omega^1 & S_R(\omega^1) &= \omega^3 & S_O(\omega^2) &= \omega^6 \\
S_L(\omega^3) &= \omega^0 & S_R(\omega^4) &= \omega^4 & S_O(\omega^5) &= \omega^7 \\
S_L(\omega^6) &= \omega^2 & S_R(\omega^7) &= \omega^5 & S_O(\omega^8) &= \omega^8
\end{aligned}
$$

For example, for $S_L(X)$, we interpolate the points:
$(\omega^0 -> \omega^1)$,
$(\omega^3 -> \omega^0)$,
$(\omega^6 -> \omega^2)$
So,

$$
\begin{aligned}
S_L(X) &= \omega^1.\ell_{L1}(X) + \omega^0.\ell_{L2}(X) + \omega^2.\ell_{L3}(X) \\
S_R(X) &= \omega^3.\ell_{R1}(X) + \omega^4.\ell_{R2}(X) + \omega^5.\ell_{R3}(X) \\
S_O(X) &= \omega^6.\ell_{O1}(X) + \omega^7.\ell_{O2}(X) + \omega^8.\ell_{O3}(X)
\end{aligned}
$$

where,
$\ell_{L1}(X)$, $\ell_{L1}(X)$, $\ell_{L1}(X)$ interpolates X at $\omega^0, \omega^3, \omega^6$ only
$S_L(X)$: evaluated/interpolated at $\omega^0, \omega^3, \omega^6$
$S_R(X)$: at $\omega^1, \omega^4, \omega^7$
$S_O(X)$: at $\omega^2, \omega^5, \omega^8$

5. Domain of Evaluation is Shared

All polynomials—wire, selector, permutation—are interpolated and evaluated at the same roots of unity. The information stored in each differs, but **the evaluation points are the same.**

Summary:

- Wire and selector polynomials encode their column values via Lagrange interpolation at the roots of unity.
- Permutation polynomials encode for each wire position which other position it is "copied from/to" via indexing, again using the roots of unity as evaluation points.

Quesitons

1. Why Not a Single Polynomial Over the Full Domain?
   ANS You could in principle define a single permutation mapping across all 99 (for n=3n=3) domain points by concatenating all the wire columns into one vector of length NN. However, PLONK's construction leverages the column structure of wire assignments for several reasons:

   Commitment Scheme Alignment: Each wire column (Left, Right, Output) is committed as a separate polynomial. This matches the construction of the permutation polynomials, which must refer specifically to the domain points associated with their wire type.

   Constraint Arithmetization: The protocol's wiring constraints operate per column—e.g., to ensure that LL at row ii and OO at row jj correspond to the same logical variable, the permutation argument must reference the Left and Output columns separately but in a coordinated way.

   Polynomial Degree Efficiency: If you interpolated all NN points with a single permutation polynomial, its degree would be higher. Using three degree-¡nn polynomials is more efficient and matches the structure of the circuit constraints (since each wire column has nn entries and degree ¡n¡n in its commitment).

   Simplicity of Proof System: By splitting the domains, each Lagrange base polynomial $\ell_k(X)\ell_k(X)$ for SL(X)SL(X), SR(X)SR(X), SO(X)SO(X) is constructed only at the corresponding positions in their domain slices, keeping each polynomial focused on its column and operation.

   In Summary

   Each permutation polynomial (SL,SR,SOSL,SR,SO) is evaluated over its own slice of the evaluation domain, reflecting the columnar structure of wire positions in the circuit.

   A single polynomial over all NN points could be defined, but it would mismatch the structure of polynomial commitments and constraint checks used in PLONK, making the protocol less modular and efficient.

   This design enforces that all positions in each wire column are addressed consistently, supporting succinct proof construction and verification.

   Key takeaway: The separation into three permutation polynomials (with each using its own Lagrange bases over wire-slice domains) is fundamental to PLONK's modularity, efficiency, and adherence to the protocol's structured circuit logic.

2. Does that mean the $L_I(X), q_L(X)$, and other selector polynomials the value and selector polynomials are also evaluated at $\omega^0, \omega^3, \omega^6$ or are they evaluated at $\omega^0, \omega^1, \omega^2$? I guess

not.

ANS Your intuition is correct: the wire assignment polynomials ($L(X), R(X), O(X)$), selector polynomials ($q_L(X), q_R(X), q_M(X), q_O(X), q_C(X)$), and similar trace polynomials are evaluated and interpolated at $\omega^0, \omega^1, \omega^2, \ldots, \omega^{n-1}\omega^0, \omega^1, \omega^2, \ldots, \omega^{n-1}$ — that is, sequentially for each gate.

For a circuit with n gates, the gate-based evaluation domain is the first n powers of the Nth root of unity, where $N = 3n$ (for 3 wires per gate), so these are $\omega^0, \omega^1, ..., \omega^{n-1}$.

Each value/selector polynomial is constructed by interpolating its column (for all gates) over this domain.

Permutation polynomials ($S_L(X), S_R(X), S_O(X)$) are the exception:

They are defined via specific subsets of the evaluation domain (e.g., S_L at ω^0, ω^3, ω^6, S_R at $\omega^1, \omega^4, \omega^7$, etc.), according to the position in the wire grid and the way variables are "flattened" in the single global wiring vector

.

In summary:

$L(X), q_L(X)$, and all other trace/selector polynomials are evaluated (and defined/interpolated) at $\omega^0, \omega^1, \omega^2, ...$ (i.e., per gate, sequentially)

.

The permutation polynomials are evaluated at their respective slices of the domain (Left, Right, Output wire slots) across all gates (i.e., every third value in the full domain for each wire column).